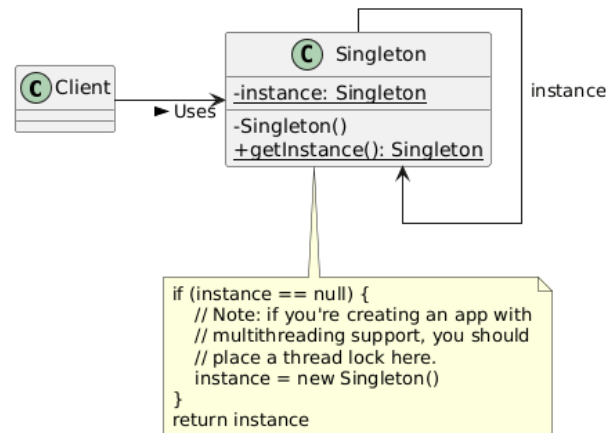


1 Singleton Pattern

Usability: Singleton pattern ensures that a class has only one instance throughout the application lifecycle and provides a global access point to it.



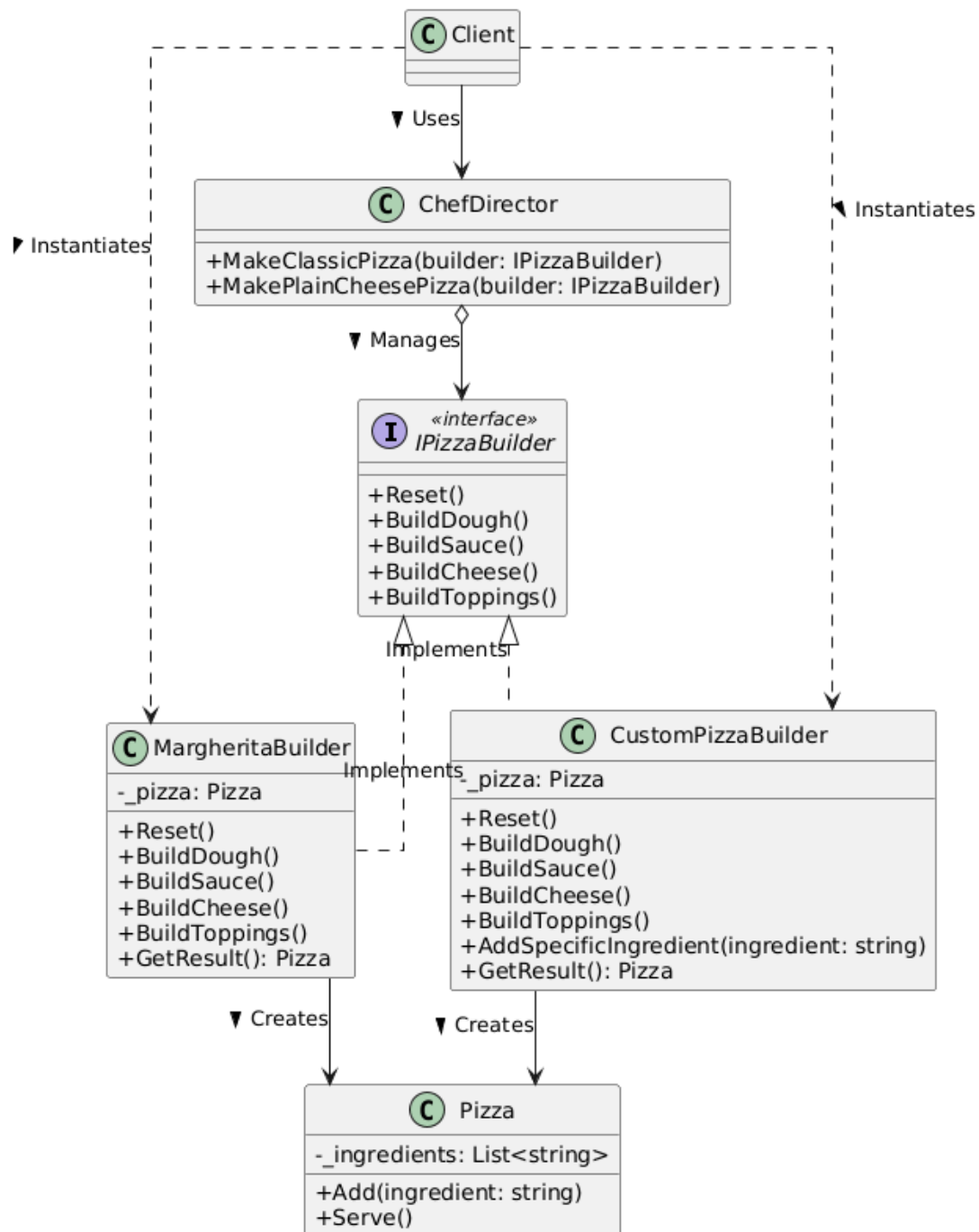
Code in C#:

```

1  using System;
2
3  public sealed class Singleton
4  {
5      private static Singleton _instance;
6      private static readonly object _lock = new object();
7      private Singleton() {
8          Console.WriteLine("Singleton initialized.");
9      }
10     public static Singleton GetInstance()
11     {
12         if (_instance == null) {
13             lock (_lock) {
14                 if (_instance == null) {
15                     _instance = new Singleton();
16                 }
17             }
18         }
19         return _instance;
20     }
21     public void DoBusinessLogic() {
22         Console.WriteLine("Executing business logic on the Singleton instance...");
23     }
24 }
25 class Program
26 {
27     static void Main()
28     {
29         Singleton instance1 = Singleton.GetInstance();
30         Singleton instance2 = Singleton.GetInstance();
31         if (ReferenceEquals(instance1, instance2)) {
32             Console.WriteLine("Success: Both variables contain the exact same memory
33             instance.");
34         } else {
35             Console.WriteLine("Failure: Instances are different.");
36         }
37         instance1.DoBusinessLogic();
38     }
39 }
  
```

2 Builder Pattern

Usability: Builder pattern separates the construction of a complex object from its representation, allowing the same step-by-step construction process to create various object configurations.



Code in C#:

```

1  using System;
2  using System.Collections.Generic;
3
4  // -- 1. Product -----
5  public class Pizza
6  {
7      private List<string> _ingredients = new List<string>();
8
9      public void Add(string ingredient)
10     {
11         _ingredients.Add(ingredient);
12     }
13
14     public void Serve()
15     {
16         Console.WriteLine($"Pizza ready with: {string.Join(", ", _ingredients)}");
17     }
18 }
19
20 // -- 2. Builder Interface -----
21 public interface IPizzaBuilder
22 {
23     void Reset();
24     void BuildDough();
25     void BuildSauce();
26     void BuildCheese();
27     void BuildToppings();
28 }
29
30 // -- 3. Concrete Builders -----
31 public class MargheritaBuilder : IPizzaBuilder
32 {
33     private Pizza _pizza = new Pizza();
34
35     public MargheritaBuilder() { this.Reset(); }
36     public void Reset() => _pizza = new Pizza();
37
38     public void BuildDough() => _pizza.Add("Thin Neapolitan Crust");
39     public void BuildSauce() => _pizza.Add("San Marzano Tomato Sauce");
40     public void BuildCheese() => _pizza.Add("Fresh Mozzarella");
41     public void BuildToppings() => _pizza.Add("Fresh Basil Leaves");
42
43     public Pizza GetResult()
44     {
45         Pizza result = _pizza;
46         this.Reset();
47         return result;
48     }
49 }
50
51 public class CustomPizzaBuilder : IPizzaBuilder
52 {
53     private Pizza _pizza = new Pizza();
54
55     public CustomPizzaBuilder() { this.Reset(); }
56     public void Reset() => _pizza = new Pizza();
57
58     public void BuildDough() => _pizza.Add("Standard Hand-Tossed Crust");
59     public void BuildSauce() => _pizza.Add("Classic Marinara");
60     public void BuildCheese() => _pizza.Add("Shredded Mozzarella");
61     public void BuildToppings() { /* Left blank on purpose for custom additions */ }
62     public void AddSpecificIngredient(string ingredient)
63     {
64         _pizza.Add(ingredient);
65     }
66
67     public Pizza GetResult()
68     {
69         Pizza result = _pizza;
70         this.Reset();
71         return result;
72     }
73 }
74
75 // -- 4. Director -----

```

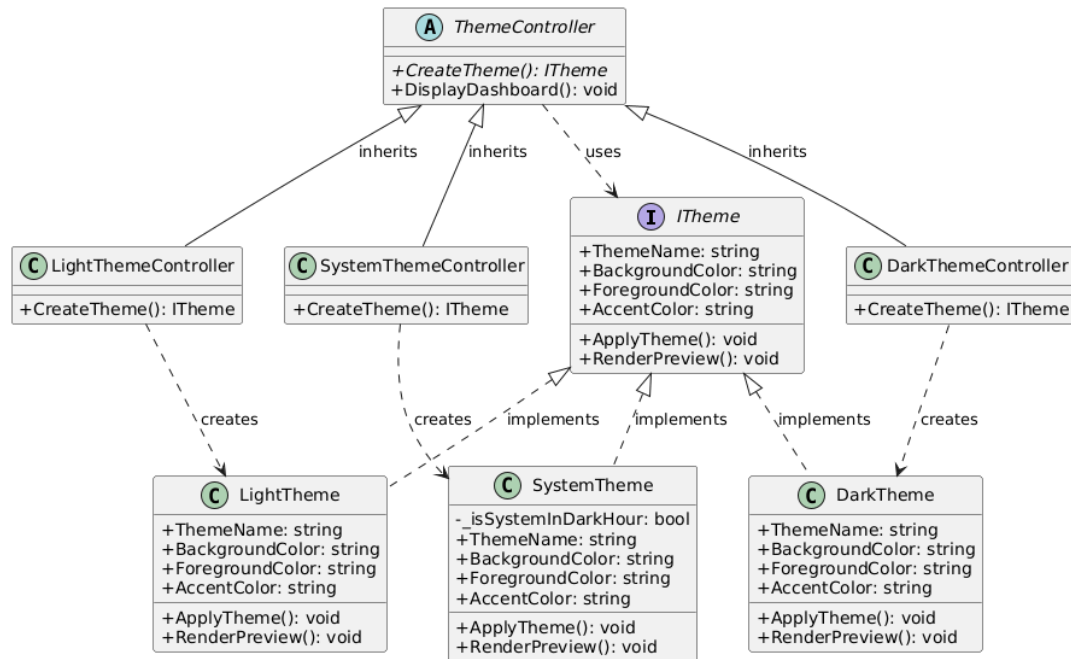
```

76 public class ChefDirector
77 {
78     public void MakeClassicPizza(IPizzaBuilder builder)
79     {
80         builder.Reset();
81         builder.BuildDough();
82         builder.BuildSauce();
83         builder.BuildCheese();
84         builder.BuildToppings();
85     }
86
87     public void MakePlainCheesePizza(IPizzaBuilder builder)
88     {
89         builder.Reset();
90         builder.BuildDough();
91         builder.BuildSauce();
92         builder.BuildCheese();
93     }
94 }
95
96 // -- 5. Client -----
97 class Program
98 {
99     static void Main(string[] args)
100     {
101         ChefDirector chef = new ChefDirector();
102
103         Console.WriteLine("—— Ordering a Classic Margherita ——");
104         MargheritaBuilder margheritaBuilder = new MargheritaBuilder();
105         chef.MakeClassicPizza(margheritaBuilder);
106         Pizza margherita = margheritaBuilder.GetResult();
107         margherita.Serve();
108
109         Console.WriteLine("\n—— Ordering a Plain Cheese Pizza ——");
110         CustomPizzaBuilder baseBuilder = new CustomPizzaBuilder();
111         chef.MakePlainCheesePizza(baseBuilder);
112         Pizza plainCheese = baseBuilder.GetResult();
113         plainCheese.Serve();
114
115         // -----
116         // NO DIRECTOR NEEDED: The client can act as the director to
117         // construct a highly customized product step-by-step!
118         // -----
119         Console.WriteLine("\n—— Building a Completely Custom Pizza ——");
120         CustomPizzaBuilder customBuilder = new CustomPizzaBuilder();
121         customBuilder.BuildDough();
122         customBuilder.BuildSauce();
123         customBuilder.BuildCheese();
124
125         // Adding custom ingredients using our specialized method
126         customBuilder.AddSpecificIngredient("Pepperoni");
127         customBuilder.AddSpecificIngredient("Mushrooms");
128         customBuilder.AddSpecificIngredient("Extra Jalapenos");
129
130         Pizza customPizza = customBuilder.GetResult();
131         customPizza.Serve();
132     }
133 }

```

3 Factory Method Pattern

Usability: Factory Method pattern defines an interface for creating an object, but allows subclasses to decide which concrete class to instantiate at runtime.



Code in C#:

```

1  using System;
2
3  // =====
4  // 1. PRODUCT INTERFACE
5  // =====
6  public interface ITheme
7  {
8      string ThemeName { get; }
9      string BackgroundColor { get; }
10     string ForegroundColor { get; }
11     string AccentColor { get; }
12
13     void ApplyTheme();
14     void RenderPreview();
15 }
16
17 // =====
18 // 2. CONCRETE PRODUCTS
19 // =====
20 public class LightTheme : ITheme
21 {
22     public string ThemeName => "Light Theme";
23     public string BackgroundColor => "#FAFAFA";
24     public string ForegroundColor => "#1F2937";
25     public string AccentColor => "#2563EB"; // Royal Blue
26
27     public void ApplyTheme()
28     {
29         Console.WriteLine($"[Light Mode] Applied: Background={BackgroundColor},
30             Text={ForegroundColor}, Accent={AccentColor}");
31     }
32
33     public void RenderPreview()
34     {
35         Console.WriteLine("-----");
36         Console.WriteLine(" | LIGHT DASHBOARD |");
37         Console.WriteLine(" | Background: Clean Slate White (#FAFAFA) |");
38         Console.WriteLine(" | Text: Deep Gray-Black (#1F2937) |");
39         Console.WriteLine(" | Button: [ Save Changes ] (Royal Blue) |");
40         Console.WriteLine("-----");
41     }
42 }
43
44 public class DarkTheme : ITheme
45 {
46     public string ThemeName => "Dark Theme";

```

```

46     public string BackgroundColor => "#0F172A";
47     public string ForegroundColor => "#F8FAFC";
48     public string AccentColor => "#38BDF8"; // Cyan Glow
49
50     public void ApplyTheme()
51     {
52         Console.WriteLine($"[Dark Mode] Applied: Background={BackgroundColor},
53             Text={ForegroundColor}, Accent={AccentColor}");
54     }
55
56     public void RenderPreview()
57     {
58         Console.WriteLine("-----");
59         Console.WriteLine($"| DARK DASHBOARD |");
60         Console.WriteLine($"| Background: Deep Midnight Navy (#0F172A) |");
61         Console.WriteLine($"| Text: Crisp Starlight White (#F8FAFC) |");
62         Console.WriteLine($"| Button: [ Save Changes ] (Neon Cyan Glow) |");
63         Console.WriteLine("-----");
64     }
65
66     public class SystemTheme : ITheme
67     {
68         private readonly bool _isSystemInDarkHour;
69
70         public SystemTheme()
71         {
72             int currentHour = DateTime.Now.Hour;
73             _isSystemInDarkHour = currentHour >= 18 || currentHour < 6;
74         }
75
76         public string ThemeName => $"System Synchronized ({(_isSystemInDarkHour ? "Dark Active"
77             : "Light Active")}");
78         public string BackgroundColor => _isSystemInDarkHour ? "#18181B" : "#F4F4F5";
79         public string ForegroundColor => _isSystemInDarkHour ? "#E4E4E7" : "#18181B";
80         public string AccentColor => "#8B5CF6";
81
82         public void ApplyTheme()
83         {
84             string mode = _isSystemInDarkHour ? "Night Schedule (Dark)" : "Day Schedule
85                 (Light)";
86             Console.WriteLine($"[System Mode] Synchronized with OS: {mode} ->
87                 Background={BackgroundColor}, Text={ForegroundColor}");
88         }
89
90         public void RenderPreview()
91         {
92             Console.WriteLine("-----");
93             Console.WriteLine($"| SYSTEM ADAPTIVE DASHBOARD |");
94             Console.WriteLine($"| Status: Syncing with OS Appearance |");
95             Console.WriteLine($"| Button: [ Save Changes ] (Adaptive Purple) |");
96             Console.WriteLine("-----");
97         }
98     }
99
100 // =====
101 // 3. ABSTRACT CREATOR (Declares the Factory Method)
102 // =====
103 public abstract class ThemeController
104 {
105     public abstract ITheme CreateTheme();
106
107     public void DisplayDashboard()
108     {
109         ITheme theme = CreateTheme();
110         Console.WriteLine($">>> Initializing: {theme.ThemeName} <<<");
111         theme.ApplyTheme();
112         theme.RenderPreview();
113     }
114 }
115
116 // =====
117 // 4. CONCRETE CREATORS
118 // =====
119 public class LightThemeController : ThemeController
120 {

```

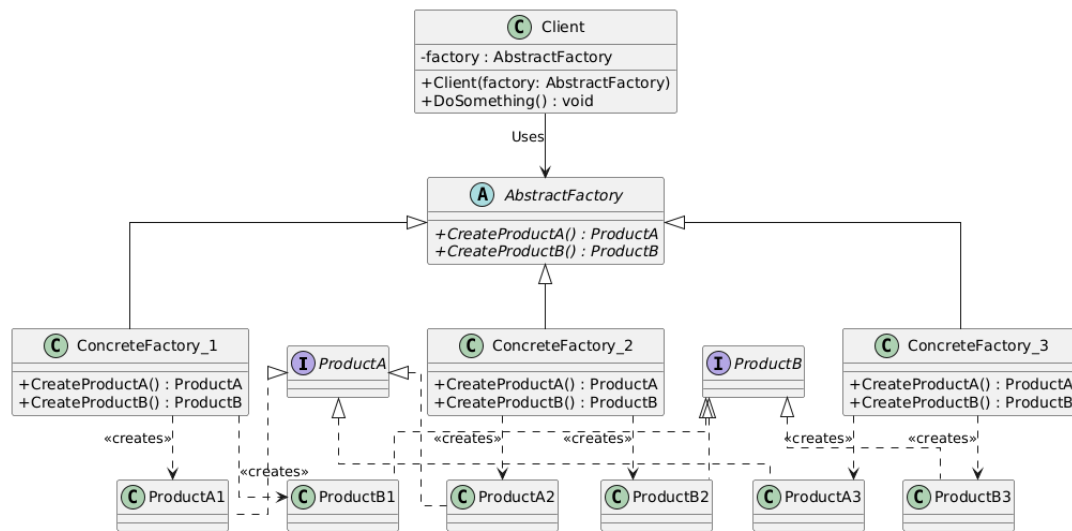
```

118     public override ITheme CreateTheme()
119     {
120         return new LightTheme();
121     }
122 }
123
124 public class DarkThemeController : ThemeController
125 {
126     public override ITheme CreateTheme()
127     {
128         return new DarkTheme();
129     }
130 }
131
132 public class SystemThemeController : ThemeController
133 {
134     public override ITheme CreateTheme()
135     {
136         return new SystemTheme();
137     }
138 }
139
140 // =====
141 // 5. CLIENT CODE
142 // =====
143 public class Program
144 {
145     public static void RenderUserInterface(ThemeController controller)
146     {
147         controller.DisplayDashboard();
148     }
149
150     public static void Main(string[] args)
151     {
152         Console.WriteLine("=====");
153         Console.WriteLine("    UI THEME ENGINE (FACTORY METHOD DEMO)    ");
154         Console.WriteLine("=====");
155
156         string[] themeModes = { "light", "dark", "system" };
157
158         foreach (var mode in themeModes)
159         {
160             ThemeController controller = mode.ToLower() switch
161             {
162                 "light" => new LightThemeController(),
163                 "dark" => new DarkThemeController(),
164                 "system" => new SystemThemeController(),
165                 _ => throw new ArgumentException("Unknown theme mode")
166             };
167
168             RenderUserInterface(controller);
169         }
170     }
171 }

```

4 Abstract Factory Pattern

Usability: Abstract Factory pattern provides an interface for creating families of related or dependent objects without specifying their concrete classes.



Code in C#:

```

1  using System;
2
3  public interface ProductA
4  {
5      void DoSomethingA();
6  }
7
8  public interface ProductB
9  {
10     void DoSomethingB();
11 }
12
13 public class ProductA1 : ProductA
14 {
15     public void DoSomethingA()
16     {
17         Console.WriteLine("Product A1 is doing its job");
18     }
19 }
20
21 public class ProductA2 : ProductA
22 {
23     public void DoSomethingA()
24     {
25         Console.WriteLine("Product A2 is doing its job");
26     }
27 }
28
29 public class ProductB1 : ProductB
30 {
31     public void DoSomethingB()
32     {
33         Console.WriteLine("Product B1 is doing its job");
34     }
35 }
36
37 public class ProductB2 : ProductB
38 {
39     public void DoSomethingB()
40     {
41         Console.WriteLine("Product B2 is doing its job");
42     }
43 }
44
45 public abstract class AbstractFactory
46 {
47     public abstract ProductA CreateProductA();
48     public abstract ProductB CreateProductB();
49 }
50
51 public class ConcreteFactory_1 : AbstractFactory
52 {

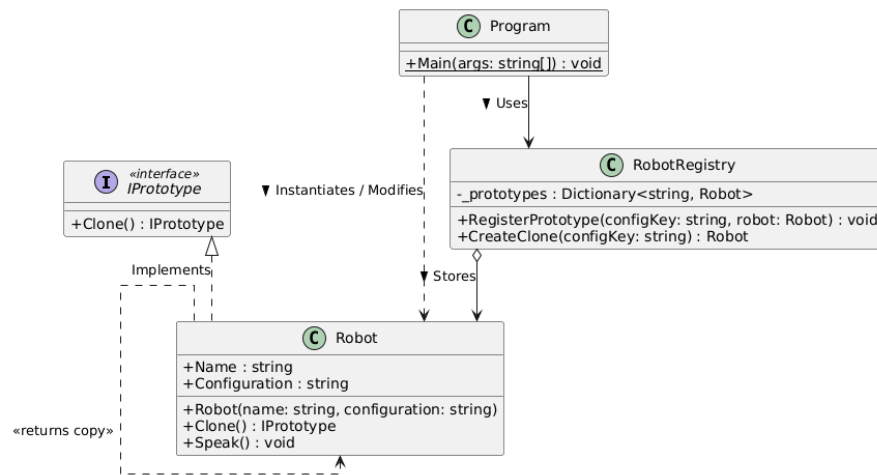
```



```
53     public override ProductA CreateProductA()
54     {
55         return new ProductA1();
56     }
57     public override ProductB CreateProductB()
58     {
59         return new ProductB1();
60     }
61 }
62
63 public class ConcreteFactory_2 : AbstractFactory
64 {
65     public override ProductA CreateProductA()
66     {
67         return new ProductA2();
68     }
69     public override ProductB CreateProductB()
70     {
71         return new ProductB2();
72     }
73 }
74
75 public class Client
76 {
77     AbstractFactory factory;
78     public Client(AbstractFactory factory)
79     {
80         this.factory = factory;
81     }
82
83     public void DoSomething()
84     {
85         ProductA productA = factory.CreateProductA();
86         ProductB productB = factory.CreateProductB();
87     }
88 }
89
90 public class Program
91 {
92     public static void Main(String[] args)
93     {
94         Client client = new Client(new ConcreteFactory_1());
95         client.DoSomething();
96
97         client = new Client(new ConcreteFactory_2());
98         client.DoSomething();
99     }
100 }
```

5 Prototype Pattern

Usability: Prototype pattern allows creating new objects by cloning an existing prototype instance instead of constructing new ones from scratch.



Code in C#:

```

1  using System;
2  using System.Collections.Generic;
3
4  public interface IPrototype
5  {
6      IPrototype Clone();
7  }
8
9  public class Robot : IPrototype
10 {
11     public string Name { get; set; }
12     public string Configuration { get; set; }
13
14     public Robot(string name, string configuration)
15     {
16         Name = name;
17         Configuration = configuration;
18         Console.WriteLine($"[Built from Scratch] Assembling {Name} with
19             {Configuration}...");
20     }
21     public IPrototype Clone()
22     {
23         Console.WriteLine($"[Cloning] Splitting/Copying {Name}...");
24         return (IPrototype)this.MemberwiseClone();
25     }
26
27     public void Speak()
28     {
29         Console.WriteLine($" -> \"HELLO, BRO!\" I am {Name} running {Configuration}.");
30     }
31 }
32
33 public class RobotRegistry
34 {
35     private Dictionary<string, Robot> _prototypes = new Dictionary<string, Robot>();
36
37     public void RegisterPrototype(string configKey, Robot robot)
38     {
39         _prototypes[configKey] = robot;
40     }
41
42     public Robot CreateClone(string configKey)
43     {
44         if (_prototypes.TryGetValue(configKey, out Robot prototype))
45         {
46             return (Robot)prototype.Clone();
47         }
48
49         throw new ArgumentException("Configuration not found in registry.");
50     }
51 }
52
53 class Program

```

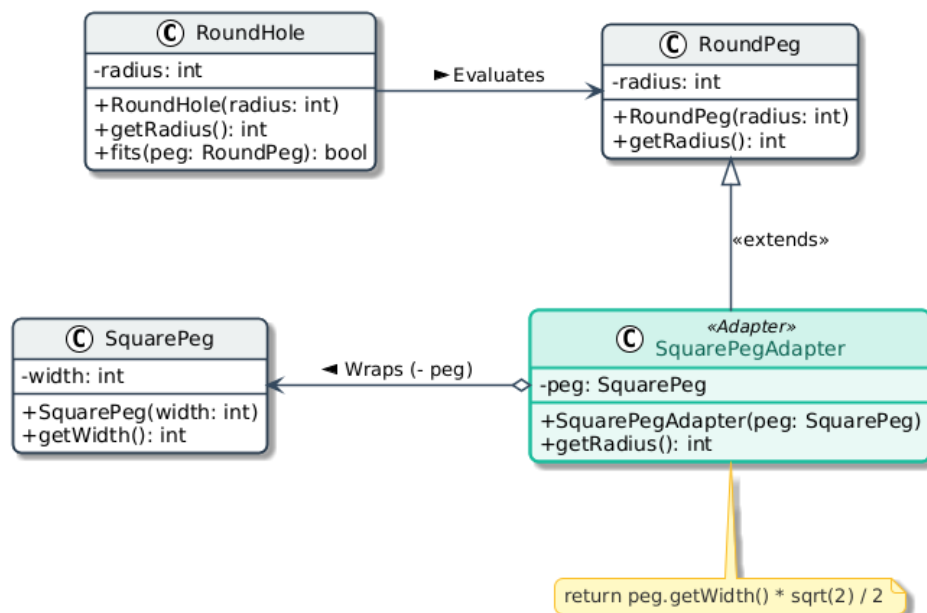
```

54 {
55     static void Main()
56     {
57         Console.WriteLine("—— INITIALIZING INCUBATORS (Setting up Prototypes) ——");
58         RobotRegistry registry = new RobotRegistry();
59
60         registry.RegisterPrototype("Config A", new Robot("Alpha-Bot", "Configuration A"));
61         registry.RegisterPrototype("Config B", new Robot("Beta-Bot", "Configuration B"));
62         registry.RegisterPrototype("Config C", new Robot("Gamma-Bot", "Configuration C"));
63
64         Console.WriteLine("\n—— CLONING PROCESS ——");
65         Robot clone1 = registry.CreateClone("Config A");
66         clone1.Name = "Alpha-Clone-1";
67         clone1.Speak();
68
69         Robot clone2 = registry.CreateClone("Config B");
70         clone2.Name = "Beta-Clone-1";
71         clone2.Speak();
72
73         Robot clone3 = registry.CreateClone("Config B");
74         clone3.Name = "Beta-Clone-2";
75         clone3.Speak();
76
77         Console.WriteLine("\n—— MEMORY CHECK ——");
78         Console.WriteLine($"Is clone2 exact same reference as clone3?
79             {ReferenceEquals(clone2, clone3)}");
80     }

```

6 Adapter Pattern

Usability: Adapter pattern converts the interface of an existing class into another interface that clients expect, allowing incompatible classes to collaborate seamlessly.



Code in C#:

```

1 using System;
2
3 // -- 1. The Target -----
4 public class RoundPeg
5 {
6     private int _radius;
7

```

```

8      public RoundPeg() { }
9
10     public RoundPeg(int radius)
11     {
12         _radius = radius;
13     }
14
15     public virtual int GetRadius()
16     {
17         return _radius;
18     }
19 }
20
21 // -- 2. The Client -----
22 public class RoundHole
23 {
24     private int _radius;
25
26     public RoundHole(int radius)
27     {
28         _radius = radius;
29     }
30
31     public int GetRadius()
32     {
33         return _radius;
34     }
35
36     public bool Fits(RoundPeg peg)
37     {
38         return this.GetRadius() >= peg.GetRadius();
39     }
40 }
41
42 // -- 3. The Adaptee (Incompatible Class) -----
43 public class SquarePeg
44 {
45     private int _width;
46
47     public SquarePeg(int width)
48     {
49         _width = width;
50     }
51
52     public int GetWidth()
53     {
54         return _width;
55     }
56 }
57
58 // -- 4. The Adapter -----
59 public class SquarePegAdapter : RoundPeg
60 {
61     private SquarePeg _peg;
62
63     public SquarePegAdapter(SquarePeg peg)
64     {
65         _peg = peg;
66     }
67
68     public override int GetRadius()
69     {
70         double result = _peg.GetWidth() * Math.Sqrt(2) / 2;
71         return (int)result;
72     }
73 }
74
75 // -- Execution -----
76 class Program
77 {
78     static void Main(string[] args)
79     {
80         Console.WriteLine("—— Testing Pegs & Holes ——");
81
82         RoundHole hole = new RoundHole(5);
83         RoundPeg rPeg = new RoundPeg(5);

```

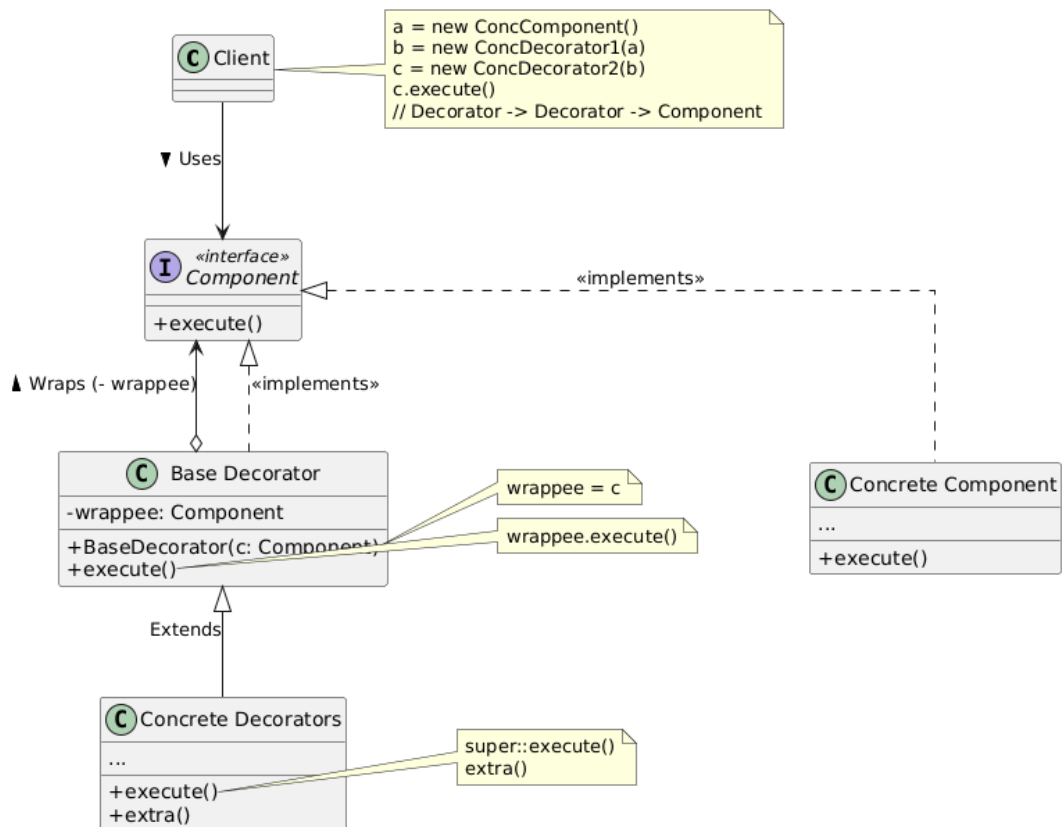
```

84
85     Console.WriteLine($"Does the round peg (r:5) fit in the hole (r:5)?
      {hole.Fits(rPeg)}");
86
87     SquarePeg smallSqPeg = new SquarePeg(5);
88     SquarePeg largeSqPeg = new SquarePeg(10);
89
90     SquarePegAdapter smallAdapter = new SquarePegAdapter(smallSqPeg);
91     SquarePegAdapter largeAdapter = new SquarePegAdapter(largeSqPeg);
92
93     Console.WriteLine($"Does the small square peg (w:5) fit?
      {hole.Fits(smallAdapter)}");
94     Console.WriteLine($"Does the large square peg (w:10) fit?
      {hole.Fits(largeAdapter)}");
95 }
96 }

```

7 Decorator Pattern

Usability: Decorator pattern dynamically attaches additional responsibilities and behaviors to an object without modifying its core class structure.



Code in C#:

```

1  using System;
2
3  // -- 1. Component Interface -----
4  public interface IComponent
5  {
6      void Execute();
7  }
8
9  // -- 2. Concrete Component -----
10 public class ConcreteComponent : IComponent
11 {

```

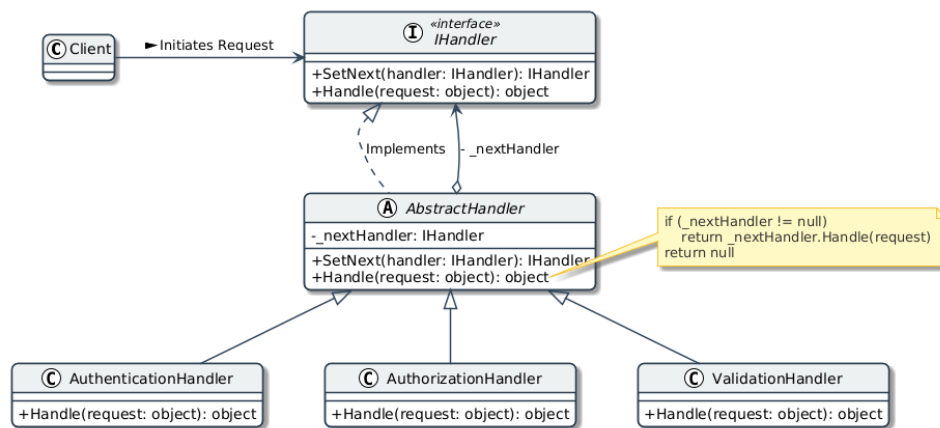
```

12     public void Execute()
13     {
14         Console.WriteLine("ConcreteComponent: Executing core behavior.");
15     }
16 }
17
18 // -- 3. Base Decorator -----
19 public abstract class BaseDecorator : IComponent
20 {
21     protected IComponent _wrappee;
22
23     public BaseDecorator(IComponent c)
24     {
25         _wrappee = c;
26     }
27
28     public virtual void Execute()
29     {
30         if (_wrappee != null)
31         {
32             _wrappee.Execute();
33         }
34     }
35 }
36
37 // -- 4. Concrete Decorators -----
38 public class ConcreteDecorator1 : BaseDecorator
39 {
40     public ConcreteDecorator1(IComponent c) : base(c) { }
41
42     public override void Execute()
43     {
44         base.Execute();
45         Extra();
46     }
47
48     private void Extra()
49     {
50         Console.WriteLine("ConcreteDecorator1: Adding extra behavior 1.");
51     }
52 }
53
54 public class ConcreteDecorator2 : BaseDecorator
55 {
56     public ConcreteDecorator2(IComponent c) : base(c) { }
57
58     public override void Execute()
59     {
60         base.Execute();
61         Extra();
62     }
63
64     private void Extra()
65     {
66         Console.WriteLine("ConcreteDecorator2: Adding extra behavior 2.");
67     }
68 }
69
70 // -- 5. Client -----
71 class Program
72 {
73     static void Main(string[] args)
74     {
75         IComponent a = new ConcreteComponent();
76         IComponent b = new ConcreteDecorator1(a);
77         IComponent c = new ConcreteDecorator2(b);
78
79         Console.WriteLine("—— Executing the decorated chain ——");
80         c.Execute();
81     }
82 }

```

8 Chain of Responsibility Pattern

Usability: Chain of Responsibility pattern passes a request along a chain of potential handlers until one of them handles it or halts invalid processing.



Code in C#:

```

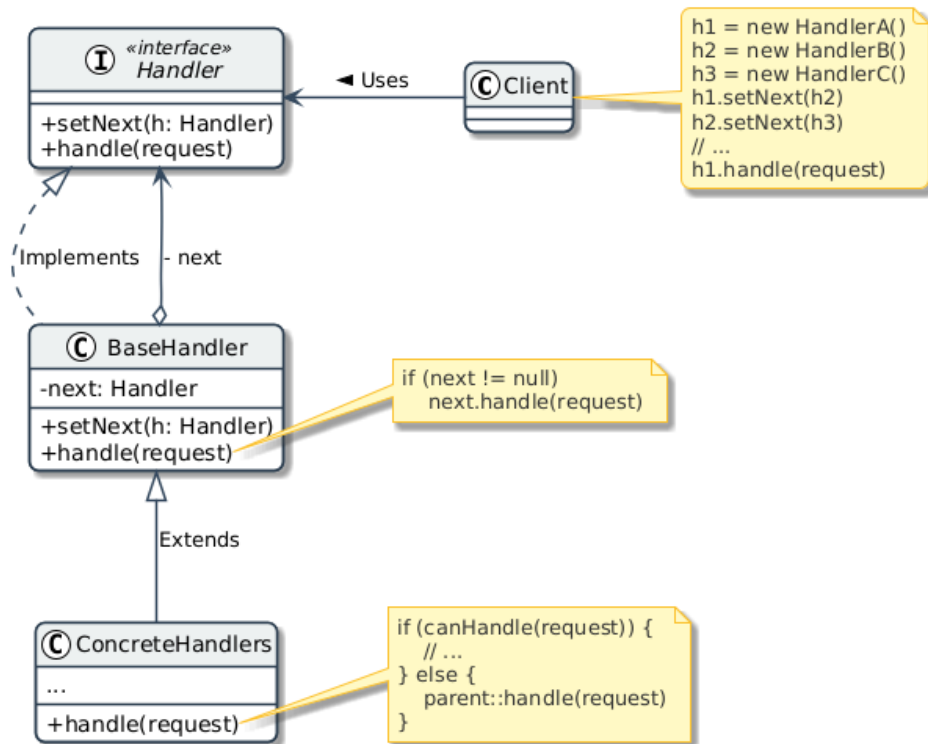
1  using System;
2
3  public interface IHandler
4  {
5      IHandler SetNext(IHandler handler);
6      object Handle(object request);
7  }
8
9  public abstract class AbstractHandler : IHandler
10 {
11     private IHandler _nextHandler;
12
13     public IHandler SetNext(IHandler handler)
14     {
15         _nextHandler = handler;
16         return handler;
17     }
18
19     public virtual object Handle(object request)
20     {
21         if (_nextHandler != null)
22         {
23             return _nextHandler.Handle(request);
24         }
25         return null;
26     }
27 }
28
29 public class AuthenticationHandler : AbstractHandler
30 {
31     public override object Handle(object request)
32     {
33         if (request.ToString() == "UnauthenticatedUser")
34         {
35             return "[Error] AuthenticationHandler: Request denied. User is not logged in.";
36         }
37
38         Console.WriteLine("AuthenticationHandler: User verified. Passing to next handler...");
39         return base.Handle(request);
40     }
41 }
42
43 public class AuthorizationHandler : AbstractHandler
44 {
45     public override object Handle(object request)
46     {
47         if (request.ToString() == "UnauthorizedUser")
48         {

```

```
49         return "[Error] AuthorizationHandler: Request denied. User lacks admin
           privileges.";
50     }
51
52     Console.WriteLine(" AuthorizationHandler: Permissions verified. Passing to next
           handler...");
53     return base.Handle(request);
54 }
55 }
56
57 public class ValidationHandler : AbstractHandler
58 {
59     public override object Handle(object request)
60     {
61         if (request.ToString() == "InvalidPayload")
62         {
63             return "[Error] ValidationHandler: Request denied. Data payload is malformed.";
64         }
65
66         Console.WriteLine(" ValidationHandler: Data validated. Passing to next handler...");
67         return base.Handle(request);
68     }
69 }
70
71 class Program
72 {
73     static void Main(string[] args)
74     {
75         var authentication = new AuthenticationHandler();
76         var authorization = new AuthorizationHandler();
77         var validation = new ValidationHandler();
78
79         authentication.SetNext(authorization).SetNext(validation);
80
81         Console.WriteLine("—— Scenario 1: Valid Request ——");
82         var result1 = authentication.Handle("ValidRequest");
83         if (result1 == null)
84         {
85             Console.WriteLine("SUCCESS: Request reached the Ordering System!");
86         }
87
88         Console.WriteLine("\n—— Scenario 2: Unauthorized Request ——");
89         var result2 = authentication.Handle("UnauthorizedUser");
90         if (result2 != null)
91         {
92             Console.WriteLine(result2);
93         }
94     }
95 }
```

9 Chain of Responsibility Pattern

Usability: Chain of Responsibility pattern decouples the sender of a request from its receivers by allowing multiple objects a chance to process the request sequentially.



Code in C#:

```

1  using System;
2
3  public interface IHandler
4  {
5      IHandler SetNext(IHandler h);
6      void Handle(string request);
7  }
8
9  public abstract class BaseHandler : IHandler
10 {
11     private IHandler _next;
12
13     public IHandler SetNext(IHandler h)
14     {
15         _next = h;
16         return h;
17     }
18
19     public virtual void Handle(string request)
20     {
21         if (_next != null)
22         {
23             _next.Handle(request);
24         }
25         else
26         {
27             Console.WriteLine($"[End of Chain] Request '{request}' was left unhandled.");
28         }
29     }
30 }
31
32 public class HandlerA : BaseHandler
33 {
34     public override void Handle(string request)
35     {
36         if (CanHandle(request))
37         {
38             Console.WriteLine($"[Handler A] Successfully processed request: {request}");
39         }
40         else
41         {
42             Console.WriteLine("[Handler A] Cannot handle. Passing to next...");
43         }
44     }
45 }

```

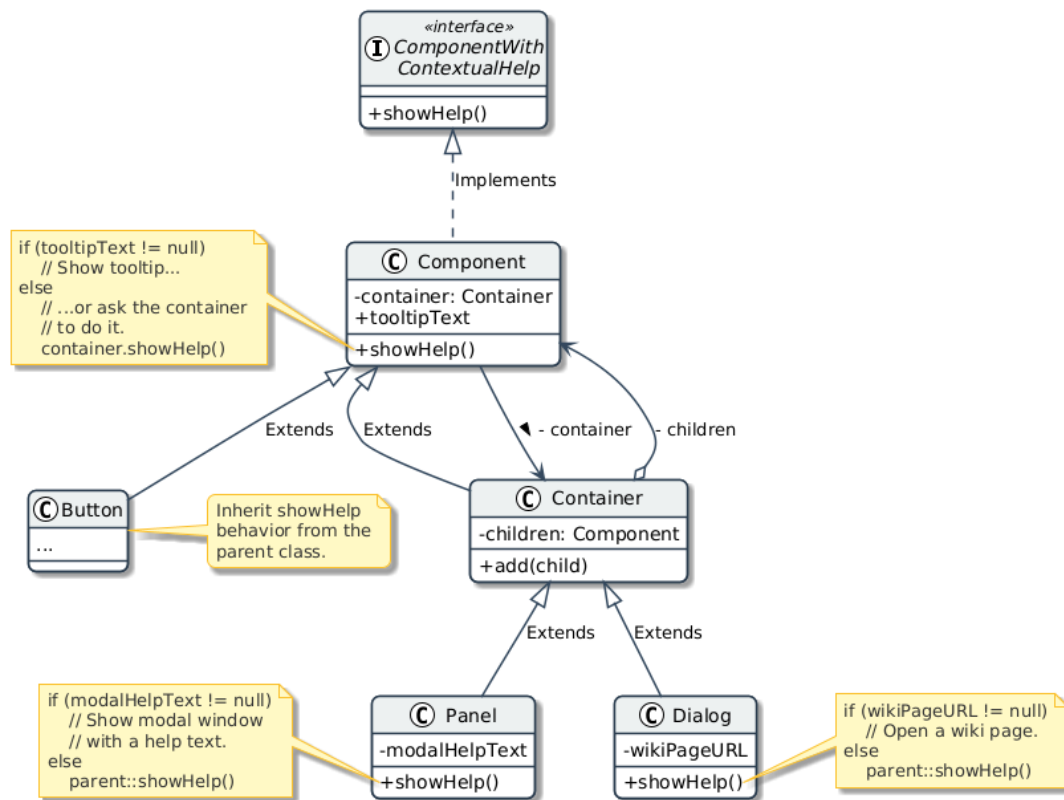
```

43         base.Handle(request);
44     }
45 }
46
47     private bool CanHandle(string request) => request == "A";
48 }
49
50 public class HandlerB : BaseHandler
51 {
52     public override void Handle(string request)
53     {
54         if (CanHandle(request))
55         {
56             Console.WriteLine($"[Handler B] Successfully processed request: {request}");
57         }
58         else
59         {
60             Console.WriteLine("[Handler B] Cannot handle. Passing to next...");
61             base.Handle(request);
62         }
63     }
64
65     private bool CanHandle(string request) => request == "B";
66 }
67
68 public class HandlerC : BaseHandler
69 {
70     public override void Handle(string request)
71     {
72         if (CanHandle(request))
73         {
74             Console.WriteLine($"[Handler C] Successfully processed request: {request}");
75         }
76         else
77         {
78             Console.WriteLine("[Handler C] Cannot handle. Passing to next...");
79             base.Handle(request);
80         }
81     }
82
83     private bool CanHandle(string request) => request == "C";
84 }
85
86 class Program
87 {
88     static void Main(string[] args)
89     {
90         Console.WriteLine("—— Building the Chain ——");
91         IHandler h1 = new HandlerA();
92         IHandler h2 = new HandlerB();
93         IHandler h3 = new HandlerC();
94
95         h1.SetNext(h2).SetNext(h3);
96
97         Console.WriteLine("\n—— Sending Request 'B' ——");
98         h1.Handle("B");
99
100        Console.WriteLine("\n—— Sending Request 'C' ——");
101        h1.Handle("C");
102
103        Console.WriteLine("\n—— Sending Request 'Z' (Unhandled) ——");
104        h1.Handle("Z");
105    }
106 }

```

10 Chain of Responsibility Pattern

Usability: Chain of Responsibility pattern routes contextual help requests upward through a hierarchical UI component tree until an element provides the help.



Code in C#:

```

1 using System;
2 using System.Collections.Generic;
3
4 public interface IComponentWithContextualHelp
5 {
6     void ShowHelp();
7 }
8
9 public abstract class Component : IComponentWithContextualHelp
10 {
11     public Container ParentContainer { get; set; }
12     public string TooltipText { get; set; }
13     public string Name { get; set; }
14
15     public virtual void ShowHelp()
16     {
17         if (TooltipText != null)
18         {
19             Console.WriteLine($"[Component: {Name}] Showing Tooltip: {TooltipText}");
20         }
21         else if (ParentContainer != null)
22         {
23             Console.WriteLine($"[Component: {Name}] No tooltip found. Passing request to parent container...");
24             ParentContainer.ShowHelp();
25         }
26         else
27         {
28             Console.WriteLine($"[Component: {Name}] Reached top of chain. No help available.");
29         }
30     }
31 }
32
33 public class Container : Component
34 {
35     protected List<Component> _children = new List<Component>();
36
37     public void Add(Component child)
38     {
39         _children.Add(child);
40     }
41 }
42
43 public class Dialog : Container
44 {
45     private string _wikiPageURL;
46
47     public string WikiPageURL { get; set; }
48
49     public void ShowHelp()
50     {
51         if (WikiPageURL != null)
52         {
53             Console.WriteLine($"[Dialog: {Name}] Opening wiki page: {WikiPageURL}");
54         }
55         else
56         {
57             Console.WriteLine($"[Dialog: {Name}] No wiki page found. Passing request to parent container...");
58             ParentContainer.ShowHelp();
59         }
60     }
61 }
  
```

```

40         child.ParentContainer = this;
41     }
42 }
43
44 public class Button : Component
45 {
46     public Button(string name) { Name = name; }
47 }
48
49 public class Panel : Container
50 {
51     public string ModalHelpText { get; set; }
52
53     public Panel(string name) { Name = name; }
54
55     public override void ShowHelp()
56     {
57         if (ModalHelpText != null)
58         {
59             Console.WriteLine($"[Panel: {Name}] Showing Modal Help: {ModalHelpText}");
60         }
61         else
62         {
63             Console.WriteLine($"[Panel: {Name}] No modal help found. Passing to parent...");
64             base.ShowHelp();
65         }
66     }
67 }
68
69 public class Dialog : Container
70 {
71     public string WikiPageURL { get; set; }
72
73     public Dialog(string name) { Name = name; }
74
75     public override void ShowHelp()
76     {
77         if (WikiPageURL != null)
78         {
79             Console.WriteLine($"[Dialog: {Name}] Opening Wiki Page URL: {WikiPageURL}");
80         }
81         else
82         {
83             Console.WriteLine($"[Dialog: {Name}] No wiki URL found. Passing to parent...");
84             base.ShowHelp();
85         }
86     }
87 }
88
89 class Program
90 {
91     static void Main(string[] args)
92     {
93         Dialog appDialog = new Dialog("Main App Dialog") { WikiPageURL =
94             "http://wiki.company.com/app-help" };
95
96         Panel settingsPanel = new Panel("Settings Panel");
97         Panel advancedPanel = new Panel("Advanced Panel") { ModalHelpText = "Warning:
98             Advanced settings can break things." };
99
100         Button okButton = new Button("OK Button");
101         Button saveButton = new Button("Save Button") { ToolTipText = "Saves your current
102             progress." };
103
104         appDialog.Add(settingsPanel);
105         appDialog.Add(advancedPanel);
106
107         settingsPanel.Add(okButton);
108         advancedPanel.Add(saveButton);
109
110         Console.WriteLine("—— Scenario 1: Button has its own help ——");
111         saveButton.ShowHelp();
112
113         Console.WriteLine("\n—— Scenario 2: Button asks Panel, Panel handles it ——");
114         advancedPanel.Add(okButton);
115         okButton.ShowHelp();

```

```
113
114     Console.WriteLine("\n—— Scenario 3: Request bubbles all the way to the top Dialog
        ——");
115     settingsPanel.Add(okButton);
116     okButton.ShowHelp();
117 }
118 }
```
